TimetablePro: Architecture & Workflow Guide

Two-Tier Scheduling Engine, Calendar Ingestion & Exam Lecture Suppression

1. Executive Summary & Why These Changes Were Made

Conventional automated timetable software operates under a simplified assumption: it solves a single, static week (Monday through Friday, Periods 1 to 6) and assumes this pattern can repeat without modification across the entire academic session. In actual educational institutions, however, a real semester runs for 16 to 18 calendar weeks, during which two very distinct types of calendar events take place:

- **Public Holidays & Vacations (e.g., Diwali break, Gandhi Jayanti):** The institution is physically closed. The scheduled classes are lost, creating syllabus completion deficits. Faculty and students require compensatory / make-up classes to cover the missing curriculum.
- **Scheduled Examinations (e.g., Internal Assessments, MSE, ESE):** The college is open, but regular teaching cannot happen because faculty are invigilating, classrooms and labs are occupied by tests, and students are writing papers.

The Software Flaw in Existing Solutions: If an administrator marked exam days as 'Closures' (holidays), traditional algorithms panicked and attempted to reschedule dozens of unnecessary make-up lectures for exam periods. Conversely, if marked as 'Open Days', regular lectures and labs were blindly scheduled at the exact same time as exams. Our new architecture resolves this cleanly.

2. The Two-Tier Architecture & Data Flow

To achieve both optimal performance and real-world flexibility, the system separates timetable generation into two distinct, cooperating engines:

Architecture Tier	Engine & Code Component	Responsibility & Database Target
Level 1: Master Weekly Blueprint	**solver.py** (Google OR-Tools CP-SAT)	Solves the ideal recurring weekly pattern without calendar dates. Stores day-of-week slots in the **timetable** database table.
Level 1.5: Calendar Ingestion	**calendar.py** (FastAPI & SQLAlchemy)	Ingests college circulars, regional public holidays, and custom Excel/CSV/PDF files. Stores events in **calendar_event** with closure and suppression flags.
Level 2: Date-Aware Semester Expansion	**semester_optimize.py** & **semester_solver.py**	Maps weekly template across 18 real calendar weeks. Suppresses lectures on exam dates and reschedules holiday make-ups into vacant slots. Stores revisions in **timetable_version** and **scheduled_session**.

3. Step-by-Step System Workflow

3.1 Academic Calendar Ingestion & Event Tagging

Events are imported from the official circular or uploaded via file. Every event is tagged with two operational flags:

Event Category	is_closure	suppresses_lectures	Resulting Scheduler Behavior

Public Holiday / Festival (e.g., Diwali, Dasara)	**True**	**True**	College closed. Classes lost. Automated search finds vacant periods for make-up lectures.
Internal Assessment / Exams (e.g., MSE, IA, ESE)	**False**	**True**	College open. Regular lectures suppressed on those dates. **NO make-up classes generated**.
Normal Teaching Day	False	False	College open. Standard weekly timetable runs normally.
Special Working Saturday	False	False (override)	Weekend override. Provides additional teaching capacity for makeup lectures.

3.2 What solver.py Generates and Where It Goes

solver.py is the Combinatorial Constraint Solver. It accepts global teacher preferences, classroom capacities, lab consecutive slot requirements, and max daily teaching limits. It executes CP-SAT to output a conflict-free, 1-week master template. This data is persisted in the PostgreSQL table **timetable** with fields: class_id, subject_id, teacher_id, room_id, and slot_id. It takes less than 2 seconds to generate.

3.3 How semester_solver.py and semester_optimize.py Process the Data

The semester orchestrator takes the template from **timetable** and executes a 3-step date expansion across the 18 weeks:

1. **Calendar Projection:** Every date in the term is paired with its corresponding weekday template row.
2. **Assessment Day Interception:** If a date has suppresses_lectures = True (e.g., Nov 16-21 for IA), regular lectures are cancelled with the outcome note *'Suppressed due to Internal Assessment (No lectures scheduled)'*. They are not flagged as missed curriculum, so no make-up lectures are required.
3. **Holiday Make-Up Rescheduling:** For actual holidays (is_closure = True), the engine searches for vacant slots:
   • *Priority 1 (Admin Preference):* If the HOD specified a preferred replacement date/period, the engine validates that slot.
   • *Priority 2 (Autonomous 35-Day Search):* If unspecified, the solver searches nearby weeks for a vacant period where teacher, class, and room are free, daily teacher workload is within limits, and the candidate date is neither an exam day nor a holiday.

4. Key Takeaways & Defense Points for Oral Presentation

Q: Why not use a single solver for the whole semester?

A single solver trying to schedule 18 weeks at once has an astronomical search space (over 10^12 variables) and would take hours to solve or time out. Decoupling the weekly master template (solver.py) from date-aware calendar expansion (semester_solver.py) gives instantaneous master solving (<2s) and instant date adjustments (<1s).

Q: Where is the data stored?

The weekly template lives in the **timetable** table. The full dated semester timetable lives in **scheduled_session**, grouped under version-controlled records in **timetable_version** with an auditable draft/publish workflow.

Q: Why was suppresses_lectures introduced instead of using is_closure?

is_closure represents an institution shutdown, which triggers automatic makeup compensation. Exams and Internal Assessments are normal college working days where lectures are intentionally suppressed. The new flag cleanly separates the two concepts with zero side-effects.

Q: How does the UI represent this?

The Weekly Timetable Grid shows red banners for 'Holiday - No Classes', amber badges for 'Exam / IA - No Lectures', and gold badges for '■ Makeup' classes.